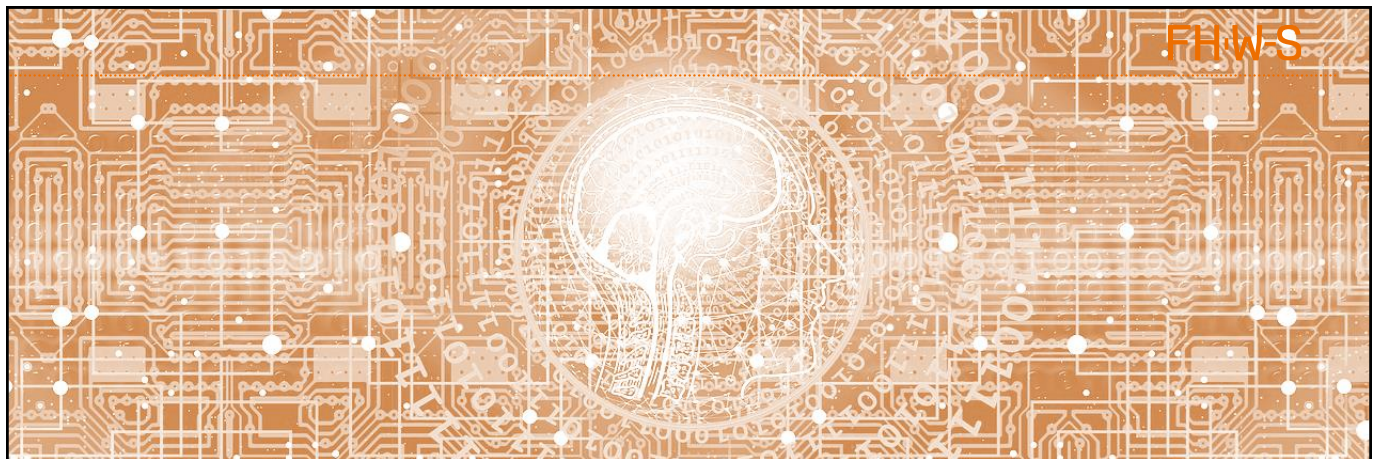




Software Engineering and Cyber Security

Prof. Dr. Marian Daun

FHWS University of Applied Sciences
Würzburg-Schweinfurt

The slide has a background image of a wooden desk with an open notebook, a pen, a smartphone, and a pair of glasses. A semi-transparent white box with an orange border contains the agenda items. The 'FHWS' logo is in the bottom left, and the number '2' is in the bottom right.

Agenda

- Software Engineering Fundamentals
- Requirements Engineering
- Architecture Design
- Modelling
- Development and Implementation
- Software Quality Assurance

FHWS University of Applied Sciences
Würzburg-Schweinfurt

2

Software-Qualitätssicherung

Softwarequalität

Quality Assurance

1. a planned and systematic pattern of all actions necessary to provide adequate confidence that an item or product conforms to established technical requirements
2. a set of activities designed to evaluate the process by which products are developed or manufactured.
3. the planned and systematic activities implemented within the quality system, and demonstrated as needed, to provide adequate confidence that an entity will fulfill requirements for quality.
4. part of quality management focused on providing confidence that quality requirements will be fulfilled.

[ISO 24765]

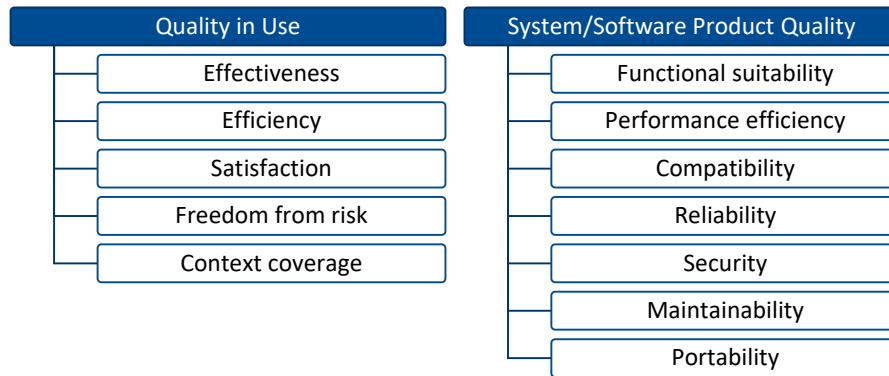
Softwarequalität

Software Quality is the

Capability of a software product to satisfy stated and implied needs when used under specified conditions.

[ISO 25010]

Qualitätsmerkmale



Basierend auf [ISO 25010]

Fehlersuche

Stark vereinfacht:

Qualitätssicherung =

Prüfung auf Fehler bezogen auf
ein Qualitätsmerkmal

Validierung und Verifikation

Verifikation: „wird das Software-System richtig entwickelt?“

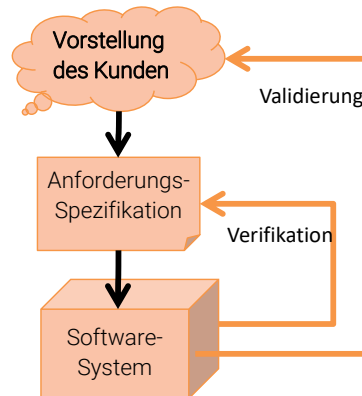
Validierung: „wird das richtige Software-System entwickelt?“

Achtung: Keine einheitliche Begriffsdefinition

Synonyme Verwendung

Oftmals: Validierung = manuell;

Verifikation = automatisiert



(stark vereinfachtes Modell der SW-Entwicklung)

Fehlerbegriff



Fehlerwirkung = Abweichung eines Systems vom geforderten/erwarteten Verhalten
Bspw. Buchhaltungssoftware liefert falschen Kontostand

failure

Kann führen zu



Fehlerzustand = Defekt in einem Software-Artefakt;
Bspw. Falsche Initialisierung einer Laufvariable (Start bei 1 statt 0).

fault

Führt zu

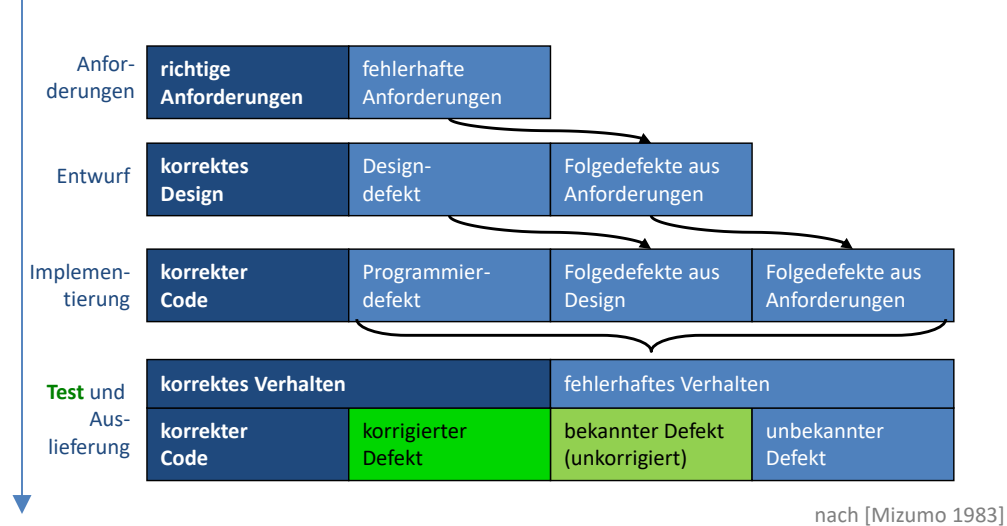


Fehlhandlung = Irrtum eines Entwicklers;
Bspw. Übersehen, dass Arrays in C++ bei 0 beginnen

error

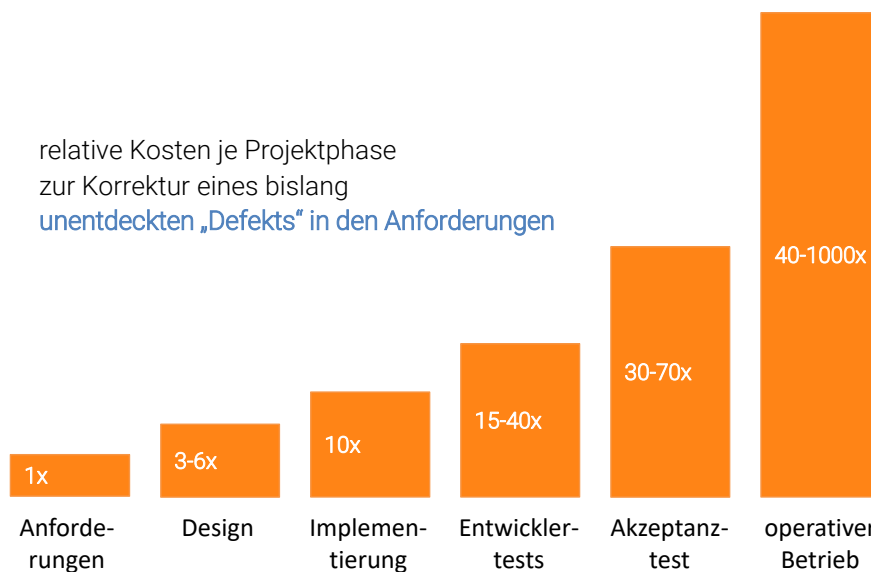
Fehler im SW-Lebenszyklus

Summierung von Fehlerzuständen ohne begleitende QS



Relative Fehlerkosten je Phase

relative Kosten je Projektphase
zur Korrektur eines bislang
unentdeckten „Defekts“ in den Anforderungen



nach [Boehm 1981]

Qualitätsmanagement

Bestandteile des Qualitätsmanagements

- Qualitätsplanung (quality planning)
- Qualitätslenkung/-steuerung (quality control)
- Qualitätsverbesserung (quality improvement)
- Qualitätssicherung (quality assurance)

in Anlehnung an [DIN EN ISO 9000:2015-11]

Qualitätsplanung

Definiert

- Qualitätsziele
- notwendige Ausführungsprozesse
- zugehörige Ressourcen
- überprüfbare Qualitätsanforderungen
- Maßnahmen zur Q-Lenkung und Q-Sicherung
- QM-Plänen

Qualitätslenkung / -steuerung

- Ziel: Sicherstellen der Qualität
- Erfüllung der Qualitätsanforderungen
- Durchführung konstruktiver Maßnahmen
Fehler vermeiden
- Durchführung analytischer Maßnahmen
Fehler finden

Konstruktive und analytische
Maßnahmen ergänzen sich

Qualitätsverbesserung

- Bewertung der aktuellen Qualitätssicherungsmaßnahmen
Messen
- Modifikationen der Q-Planung
Ziele, Qualitätsanforderungen, Planung
- Modifikation Q-Lenkung
Maßnahmen, Prozesse

Qualitätssicherung

Prüfung der

- Einhaltung der Qualitätsanforderungen
- Wirksamkeit des Qualitätsmanagementsystems
(ggf. Zertifizierungen)
- qualitätsrelevanten Messgrößen
- Prozesse und Qualitätsverfahren

Messen und Bewerten



Group Discussion

Software-Messung – Warum?

- Kontrolle
 - Softwarequalität
 - Software-Entwicklungsprozess
 - Einhaltung von Standards
- Planung
 - Aufwand, Kosten, Zeit
 - Einführungsbegleitung neuer Methoden, Werkzeuge, etc.
- Vergleich und Beurteilung
- Frühzeitige Problemidentifikation

Software-Messung

Drei Schritte

1. **Verstehen**
Bewertung der aktuellen Situation
Aufdecken von Ursache-Beziehungen
2. **Steuern**
Überwachung von Prozessen und Produkten
Änderungsentscheidungen
3. **Verbessern**
Prozess- und Produkteigenschaften

Software-Messung

Software-Maß

Definition:

Ein **Maß** ist eine **Messvorschrift**, die eine **Methode** zur Erhebung von **Daten** zur Charakterisierung eines bestimmten **Merkmals (Attributs)** des Untersuchungsgegenstandes beschreibt.

Software-Maße

Interne vs Externe Maße

- **Interne Attribute**
können direkt im Produkt/Prozess gemessen werden
bspw. Lines of Code
- **Externe Attribute**
können durch Untersuchung des "Verhaltens" bestimmt werden
bspw. Benutzerfreundlichkeit

	Interne Attribute	externe Attribute
Sichtbarkeit	früh	spät
Messbarkeit	i.d.R. einfach	schwierig (kontext-spezifisch)

- Oft werden daher **interne Attribute gemessen**, um **Aussagen über externe Attribute** machen zu können,
 - z.B. *Anzahl Anforderungen* → *Abschätzung Entwicklungsaufwand*

Validierung von Maßen

Probleme

Nachweis der Tauglichkeit eines Maßes ist schwierig

Sagt zyklomatische Komplexität eines Kontrollflussgraphen wirklich etwas über die „psychologische“ Komplexität des Artefakts aus?

Ist die Länge (LOC) von Code ein sinnvolles Maß, um die Ausdehnung zu Messen?

Validierung der Maße ist essentiell

Aber: sehr aufwändig

Wird oft nicht oder nur in geringem Umfang durchgeführt
Validierung in verschiedenen Kontexten

Beispiele für Software-Maße

Produktmaße

Beispiel 1: Lines of Code

- Einsatz zur Abschätzung des Entwicklungsaufwands
- Lines of Code (LOC)
Nachteil: je nach Anzahl der Kommentare unterschiedliche Ergebnisse
- Non-commenting Lines of Code (NLOC)
Nachteil: je nach "Stil" immer noch unterschiedliche Ergebnisse
- Non Commenting Source Statements (NCSS)
Nur Zählen Source-Code spezifischer Zeichen.
Bspw. ';' und '{' Zeichen in Java Source Files

Produktmaße

Beispiel 1: Lines of Code

Vorteile LOC/NCSS:

- Sehr einfach zu berechnen
- LOC kann jedes UNIX
- Kommando: `wc -l`
- Für NCSS existieren spezielle Tools
- <http://www.kclee.de/clemens/java/javancss/>

Nachteil:

- Das LOC/NCSS Maß ist grundsätzlich sprachabhängig
- D.h. bei Vergleich von LOC(python) mit LOC(Java) vergleicht man "Äpfel" mit "Birnen"!
- Bei LOC/NCSS geht es nur um Größe
- Nutzung dieser Maße für die Abschätzung von Produktivität, Aufwand, Komplexität etc. kann problematisch sein

Prozessmaße

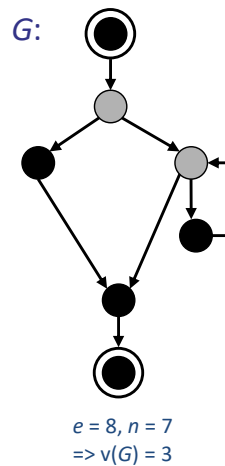
Beispiel 2: Aufwand

- Erfassung erfolgt bspw.
 - über Arbeitszeit
 - dediziert pro Aktivität (z.B. Zeit vom Checkout eines Artefakts bis zum Release)
- **Aber:** besitzt gewisse "Unschärfe"
 - Z.B. wie erfasst man die Zeit zum "Nachdenken", parallele Arbeit an mehreren Artefakten, ...
- Von Aufwand wird oft das Maß **Effizienz** oder **Produktivität** abgeleitet
 - $\text{Effizienz} = \text{Output/Input}$
 - Beispiel: $\text{Effizienz} = \text{Größe/Aufwand [LOC / Zeit]}$

Produktmaße

Beispiel 3: Zyklomatische Komplexität

- Zyklomatische Zahl / $v(G)$
- Abschätzung der internen Komplexität
- Eingeführt 1976 von Thomas McCabe
 - $v(G) = e - n + 2$
 - e : edges (Kanten)
 - n : nodes (Knoten)
 - $v(G)$ = Anzahl Entscheidungen + 1



Produktmaße

Beispiel 3: Zyklomatische Komplexität

$v(G)$	Risiko-Einschätzung
1-10	a simple program, without much risk
11-20	more complex, moderate risk
21-50	complex, high risk program
> 50	untestable program (very high risk)

[www.sei.cmu.edu]

Goal Question Metric (GQM)

Die GQM-Methode

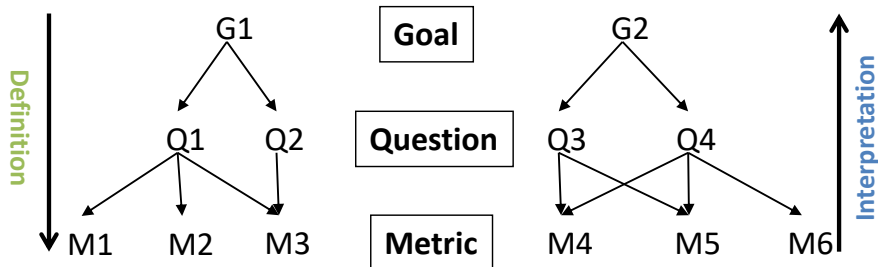
Goal / Question / Metric

- Zielgerichtete, systematische, strukturierte Definition und Interpretation von Maßen
- Nutzen:
 - Einsatz zur **Verbesserung der Software-Entwicklung** durch systematische, ziel-orientierte Messung
 - Zunehmend aber auch zur strukturierten **Definition von Experimenten** und empirischen Studien (Ableitung von Forschungsfragen und deren Quantifizierung durch Maße)

[Basili, Rombach][van Solingen, Berghout]

Die GQM-Methode

Goal / Question / Metric



- Goals definieren explizite Mess-Ziele
- Questions verfeinern die Goals
- Metrics quantifizieren die Fragen

[Basili, Weiss 1984]

Die GQM-Methode

Goals

A **goal template** supports the **structured** and **understandable definition** of measurement goals:

Analyse	the object under measurement
For the purpose of	understanding , controlling or improving the object
With respect to	the quality focus of the object under measurement
From the viewpoint of	the people that measure the object
In the context of	the environment in which measurement takes place

GQM goal definition template [Basili et al 1994]

Die GQM-Methode

Beispiel

Goal

Analyse the	delivered product and development process
For the purpose of	better understanding
With respect to	reliability and its causes
From the viewpoint of	the software development team (SW project leader and software engineer)
In the context of	project A

[van Solingen et al.]

Die GQM-Methode

Beispiel

- Q.2: Is the source code in accordance with coding standards?
M.2.1: Number of violations of coding standards
- Q.3: What is the complexity/size of the delivered software?
M.3.1: Length of code (KSLOC)
M.3.3: McCabe's cyclomatic complexity per module
M.3.5: Change rate of the module
M.3.6: Personal assessment (subjective measure) of the overall complexity per module (high, medium, low)
- Q.7: What was the distribution of failures after delivery over severity classes?
M7.1.: Percentage of faults by severity
- Q.12: What was the relationship between module complexity and reliability?
M.12.1: Correlation coefficient (Failures vs. Module Complexity)
...

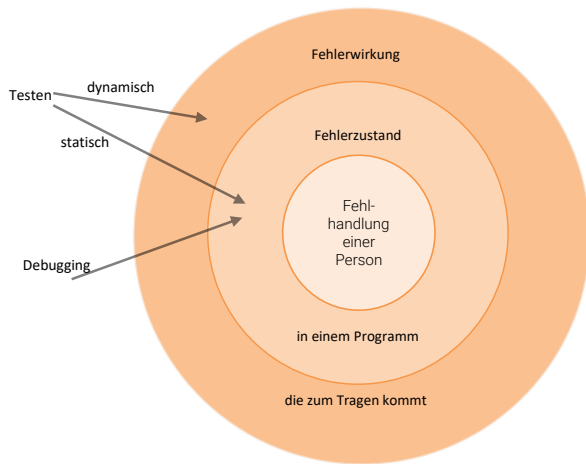
[van Solingen et al.]

Grundlagen des Softwaretestens

Testen (Dynamischer Test)

- Ausführung des Testobjekts mit dem Ziel Fehler aufzudecken
NICHT: Die Fehlerfreiheit zu beweisen
- Stichprobenartig

Fehlerbegriff



[Spillner & Linz 2019]

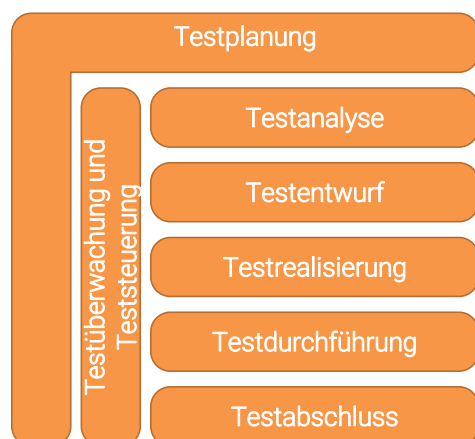
Testbegriff

1. **Testziel** Benennung anhand des Testzwecks (z.B. Lasttest)
2. **Testmethode/Testverfahren** Benennung anhand der Methode/des Verfahrens das zur Durchführung des Tests eingesetzt wird (z.B. zustandsbasierter Test)
3. **Testobjekt** Benennung anhand des Testobjekts, das getestet wird (z.B. GUI-Test)
4. **Teststufe** Benennung anhand der Stufe des zugrunde liegenden Softwareentwicklungsmodells (z.B. Systemtest)
5. **Testperson** Benennung anhand der testdurchführenden Personengruppe (z.B. Entwicklertest, Anwendertest)
6. **Testumfang** Benennung anhand des Umfangs (z.B. partieller Regressionstest).

Grundsätze des Testens

1. Testen zeigt die Anwesenheit von Fehlerzuständen
2. Vollständiges Testen ist nicht möglich
3. Frühes Testen spart Zeit und Geld
4. Häufung von Fehlerzuständen
5. Vorsicht vor dem Pestizid-Paradoxon
6. Testen ist kontextabhängig
7. Trugschluss: Keine Fehler bedeutet ein brauchbares System!

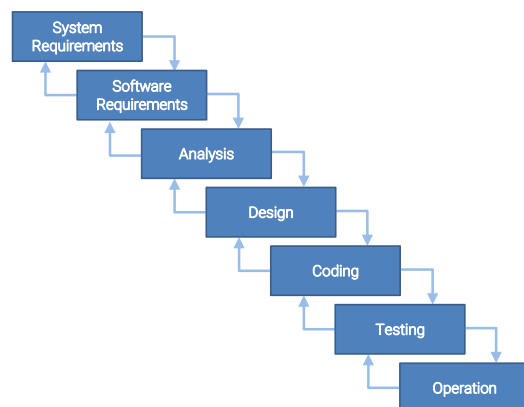
Testprozess



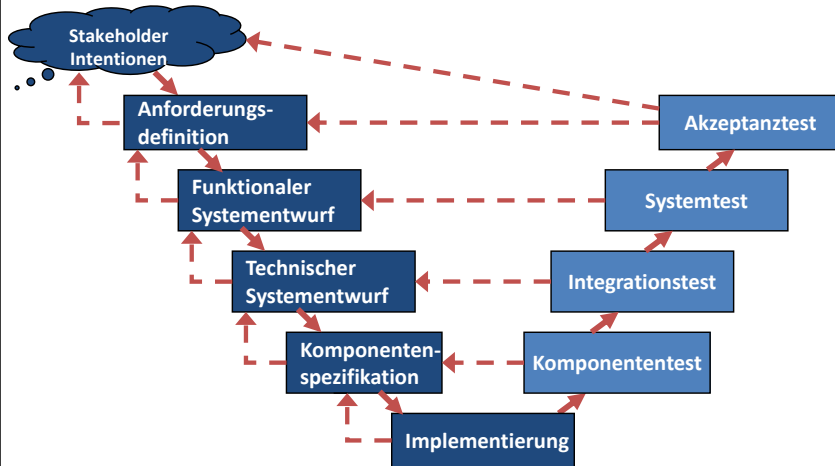
[Spillner & Linz 2019]

Testen im Softwareentwicklungslebenszyklus

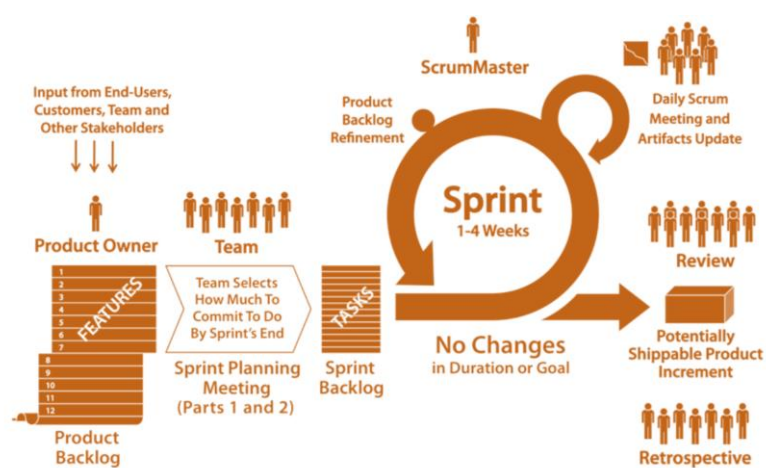
Wasserfallmodell



V-Modell



Scrum



[Sutherland 2011]

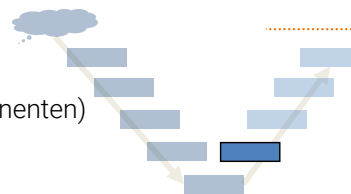
Teststufen

Komponententest

- Test der feingranularsten Softwarebausteine (Komponenten)
Häufig: Klassen, Module, Units
- Isoliertes Testen
Unabhängig von anderen Komponenten
Unabhängig von externen Faktoren
- Codenah, Entwicklertests

Testziele:

Funktionale Korrektheit
Funktionale Vollständigkeit
Qualitätsfaktoren (Robustheit, Effizienz, Wartbarkeit, ...)



Integrationstest

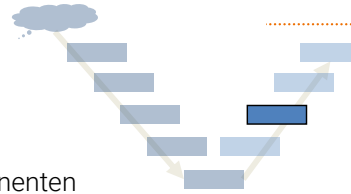
- Test von Teilsystemen
- Teilsysteme bestehen aus bereits getesteten Komponenten
- Iterative Integration

Testziele

- Schnittstellen
- Zusammenspiel zwischen Komponenten

Herausforderungen

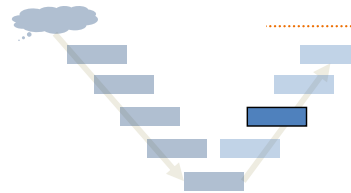
- Integration von Fremdsystemen
- Integration von zugekauften Komponenten (COTS)



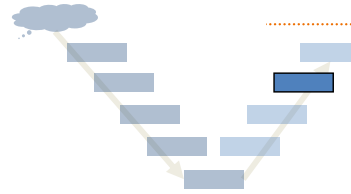
Integrationstest

Integrationsstrategien

- Top-down-Integration
- Bottom-up-Integration
- **Ad-hoc-Integration**
- Backbone-Integration



Systemtest



- Prüfung des Gesamtsystems
- Test gegen die Anforderungsspezifikation
- Systemtests werden nicht in der Produktivumgebung durchgeführt
- Systemtest werden oft unterschätzt

Abnahmetest (Akzeptanztest)



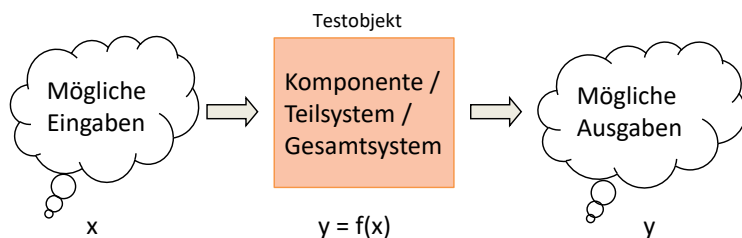
- Abnahme durch den Kunden
- Fehlersuche nicht im Fokus
- Kann auch auf niedrigeren Teststufen angewandt werden
- Typische Formen des Akzeptanztests
 - Test auf vertragliche Akzeptanz
 - Test der Benutzerakzeptanz
 - Akzeptanz durch den Systembetreiber
 - Feldtest (Alpha- und Beta-Tests)

Äquivalenzklassenbildung

Prinzip des Äquivalenzklassentests

Problem: In der Regel ist es unmöglich das Prüfobjekt mit sämtlichen Kombinationen der Eingabewerte zu testen

Lösung: Unterteilung der Eingaben in „Gruppen ähnlicher Werte“
→ Äquivalenzklassen



Äquivalenzklasse

ist eine Teilmenge von Elementen aus einer Gesamtmenge, die untereinander in einer Äquivalenzrelation stehen

Annahme:

Sämtliche Elemente einer Äquivalenzklasse führen zu **demselben Testergebnis**, d.h. entweder zu PASS oder FAIL

Folgerung:

Test eines **Repräsentanten** einer Äquivalenzklasse reicht aus

Bildung von Äquivalenzklassen

- Unterteilung aller möglichen Eingabedaten
 - in **gültige** Äquivalenzklassen
 - und **ungültige** Äquivalenzklassen (Test der Fehlerbehandlung)
- Test durch einen (oder mehrere) Repräsentanten
 - **Reduktion** der Anzahl von Testfällen.

Äquivalenz ist hypothetisch

(Klassen werden z.T. nach Erfahrung und Intuition gebildet)

- Nachvollziehbare Benennung von Zusammenhängen
 - Bspw: Gültige Äquivalenzklassen mittels Zahlen: 1, 2, ..
 - Ungültige Äquivalenzklassen dazu passend mit 1a, 1b, 2a

Bildung von Äquivalenzklassen

- **Definitionsbereich = Wertebereich**
 - Eingabevariable in geordnetem Wertebereich
 - eine gültige Äquivalenzklasse
 - zwei ungültige Äquivalenzklassen
 - Beispiel: Wertebereich $1 \leq \text{Wert} \leq 49$
 - gültig: $1 \leq \text{Wert} \leq 49$
 - ungültig: $\text{Wert} < 1, \text{Wert} > 49$
- **Definitionsbereich = Datenstrukturen (Mengengerüst)**
 - Gegebene untere und obere Schranken
 - eine gültige Äquivalenzklasse
 - zwei ungültige Äquivalenzklassen
 - Beispiel: Eine Liste enthält 1 bis 155 Elemente
 - gültig: 1 bis 155
 - ungültig: 0, > 155 Elemente

Bildung von Äquivalenzklassen

- **Definitionsbereich = Eingabevariable unterliegt Bedingungen**
 - eine gültige und eine ungültige Äquivalenzklasse bilden
 - Beispiel: Eingabe ist natürliche Zahl
 - gültige Ä-Klasse: $x \in \mathbb{N}$
 - ungültige Ä-Klasse: $x \notin \mathbb{N}$
- **Definitionsbereich = Aufzählung / ungeordneter Wertebereich**
 - eine gültige Ä-Klasse mehrere gültige Äquivalenzklassen wenn Werte ungleich behandelt
 - eine ungültige Ä-Klasse
 - Beispiel: Aufzählung: {rot, gelb, grün}
 - gültig: {rot, gelb, grün}
 - (d.h.: $\text{Wert} = \text{rot}, \text{Wert} = \text{gelb}, \text{Wert} = \text{grün}$)
 - ungültig: Wert nicht in {rot, gelb, grün}

Testfälle für Äquivalenzklassen

- Alle gültigen Äquivalenzklassen werden getestet
 - Ein Test kann mehrere Äquivalenzklassen abdecken
 - Tests werden so gruppiert, dass möglichst viele Äquivalenzklassen mit wenig Tests abgedeckt werden
- Alle ungültigen Äquivalenzklassen werden getestet
 - Jeder Test testet **genau eine** ungültige Äquivalenzklasse
 - Nachvollziehbarkeit
 - Abhängigkeit zwischen Fehlern
- Erweiterte Äquivalenzklassenbildung
 - Testfälle werden für Kombinationen von Äquivalenzklassen gebildet
 - Jede Kombination bildet Testfall

Beispiel

- Mitarbeiter erhalten ab einer Zugehörigkeit (**x**) zur Firma eine Weihnachtsgratifikation:
 - Bei mehr als **3 Jahren** -> 50% des Monatsgehalts
 - Bei mehr als **5 Jahren** -> 75% des Monatsgehalts
 - Bei mehr als **8 Jahren** -> 100% des Monatsgehalts
- Zusätzlich wird nach Gehaltsgruppen (**g**) unterschieden
 - Für Gehaltsgruppe G2 wird eine Gratifikation von 50% gewährt, für G3 100%. G1 erhält 0%

Schritt 1:

Zugehörigkeit: $0 \leq x \leq 50$
 Gehaltsgruppe: $g \in \{G1, G2, G3\}$

Parameter	gültige Äquivalenzklassen	ungültige Äquivalenzklassen
1. Zugehörigkeit	1.1: $0 \leq x \leq 3$ 1.2: $3 < x \leq 5$ 1.3: $5 < x \leq 8$ 1.4: $8 < x \leq 50$ *	1.a: $x < 0$ 1.b: $x > 50$ *
2. Gehaltsgruppe	2.1: $g = G1$ 2.2: $g = G2$ 2.3: $g = G3$	2.a: nicht in $\{G1, G2, G3\}$

* Der Wert 50 ergibt sich aus dem gewählten Definitionsbereich.

Testfall	T1	T2	T3	T4	T5	T6	T7
Zugehörigkeit	2	4	7	12	-1	83	7
Gehaltsgruppe	G1	G2	G3	G2	G1	G3	G4
abgedeckte Äquivalenzklasse	<u>1.1</u> <u>2.1</u>	<u>1.2</u> <u>2.2</u>	<u>1.3</u> <u>2.3</u>	<u>1.4</u> <u>2.2</u>	<u>1.a</u>	<u>1.b</u>	<u>2.a</u>

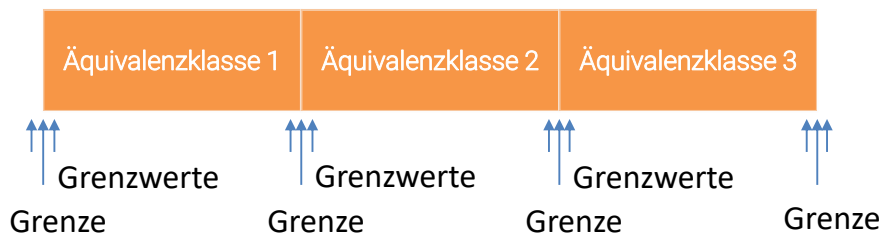
Die unterstrichenen Äquivalenzklassen werden durch den entsprechenden Testfall neu abgedeckt

Grenzwertanalyse

Prinzip der Grenzwertanalyse

Idee

- Fehler sind nicht gleichverteilt
- Häufiges Auftreten in den Grenzbereichen von Äquivalenzklassen



Bestimmung von Grenzwerten

- Für jede Grenze einer Äquivalenzklasse Testfälle für
 - den **exakten** Grenzwert
 - **Inneren** Grenzwert
 - **Äußeren** Grenzwert
- Weitere besondere Werte
 - Null
 - Positiv und negativ
 - ...
- Reduktion des Umfangs
 - Kein Test mittlerer Werte einer Äquivalenzklasse
 - Gemeinsame Grenzen zweier Äquivalenzklassen werden nur einmal getestet

Beispiel

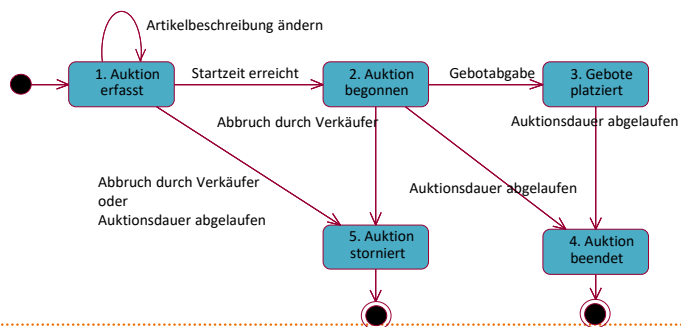
- Eingabefeld x in einer Maske ist 16-bit Integer-Zahl
- Äquivalenzklassen:
 - **gültige Äquivalenzklasse 1:** [-32768, 32767]
 - ungültige Äquivalenzklasse 1a: < -32768
 - ungültige Äquivalenzklasse 1b: > 32767
- Äquivalenzklassentestfälle:
 - z.B. 13566, -170000, 180000
- Testfälle aus Grenzwert Analyse:
 - -32769, **-32768**, -32767 (untere Grenze)
 - 32766, **32767**, 32768 (obere Grenze)
 - 0, -1, +1 (besondere Werte)

Zustandsbasierter Test

Prinzip des zustandsbasierten Tests

Idee

- Ausgaben hängen nicht nur von Eingaben ab, sondern auch von der Historie der Systemeinführung
- Modellierung der Historie mittels Zustandsmodellen

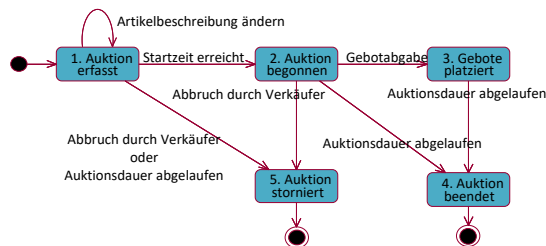


Testfallbestimmung

- **Alle Zustände**
 - Jeder Zustand muss mindestens einmal erreicht werden
 - Analog zu C_0
- **Alle Transitionen**
 - Jeder Zustandsübergang muss mind. einmal durchlaufen werden
 - Analog zu C_1
 - Subsumiert den Test aller Zustände
- **Alle Ereignisse**
 - Je Transition muss jedes Ereignis mindestens einmal ausgelöst werden
 - Analog Bedingungstest
 - Subsumiert Test aller Zustandsübergänge

Testfallableitung

- Definition von Testfällen von Startzustand bis Endzustand



- 1(S): "Startzeit erreicht" → 2: "Gebotabgabe" → 3: "Auktionsdauer abgelaufen" → 4(E)
- 1(S): "Startzeit erreicht" → 2: "Abbruch durch Verkäufer" → 5(E)
- 100% Zustandsüberdeckung

Zustandsübergangstabelle

- Zustandsübergangstabellen helfen Tests für Situationen zu definieren, die nicht eintreten dürfen
 - Kann nicht spezifizierte Transition tatsächlich nicht erfolgen?

		"ändern"	"starten"	"bieten"	"beenden"	"abbrechen"
1	Auktion erfasst	1	2	-	5	5
2	Auktion begonnen	-	-	3	4	5
3	Gebote platziert	-	-	-	4	-
4	Auktion beendet	-	-	-	-	-
5	Auktion storniert	-	-	-	-	-

Entscheidungstabellentests

Prinzip des Entscheidungstabellentests

Idee

- Eingabeparameter sind nicht unabhängig
- Ursache-Wirkung-Beziehungen müssen bei der Testspezifikation berücksichtigt werden

Lösung: Entscheidungstabellen helfen bei der Identifikation von interessanten Eingabeparameterkombinationen

- Systematische Auswahl kann auch die Anzahl der benötigten Testfälle reduzieren

Entscheidungstabelle

Entscheidungstabelle		Testfall 1	Testfall 2	Testfall 3	Testfall 4
Bedingungen	Bed. 1	ja	ja	nein	nein
	Bed. 2	ja	nein	ja	nein
	...				
Aktionen	Aktion 1	x	x	x	-
	Aktion 2	-	-	x	x
	...				

Konsolidierungen

1. Ungültige Bedingungen

- Bereinigung um sich ausschließende Bedingungen

Entscheidungstabelle		Testfall 1	Testfall 2	Testfall 3	Testfall 4
Bedingungen	Bed. 1	ja	ja	nein	nein
	Bed. 2			ja	nein
	...				
Aktionen	Aktion 1	x	x	x	-
	Aktion 2	-	-	x	x
	...				

Konsolidierungen

2. Identische Testfälle

- Identische Spalten werden entfernt

Entscheidungstabelle		Testfall 1		Testfall 3	Testfall 4
Bedingungen	Bed. 1	ja		nein	nein
	Bed. 2	-		ja	nein
	...				
Aktionen	Aktion 1	x		x	-
	Aktion 2	-		x	x
	...				

Konsolidierungen

Ergebnis

Entscheidungstabelle		Testfall 1	Testfall 3	Testfall 4
Bedingungen	Bed. 1	ja	nein	nein
	Bed. 2	-	ja	nein
	...			
Aktionen	Aktion 1	x	x	-
	Aktion 2	-	x	x
	...			

Kontrollflussbasiertes Testen

Prinzip des Kontrollflussbasierten Testens

Idee

- Quellcode wird in einen Kontrollflussgraphen überführt
- Testfälle werden aus diesem Graphen abgeleitet
- Vollständigkeit kann in Bezug auf den Kontrollflussgraphen bestimmt werden

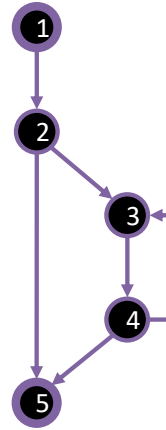
Tests

- Anweisungsüberdeckungstest
- Entscheidungsüberdeckungstest
- Bedingungsüberdeckungstest
- Pfadüberdeckungstest

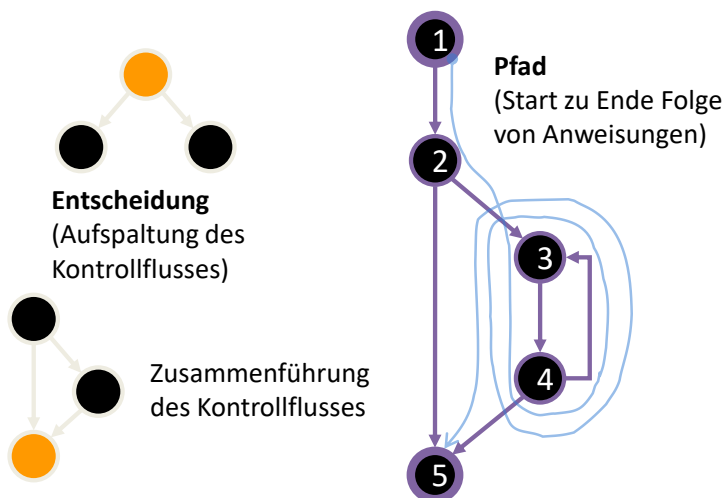
Kontrollflussgraph

- Gerichteter Graph $G = (N, E, n_{\text{start}}, n_{\text{final}})$
 - N Menge von **Knoten (Nodes)**
 - $E \subseteq N \times N$ Menge **gerichteter Kanten (Edges)**
 - $n_{\text{start}} \in N$ **Startknoten**
 - $n_{\text{final}} \in N$ **Endknoten**

- Knoten** = Anweisung im Programmcode
- Kante** = Kontrollfluss von einer Anweisung zur Nächsten



Terminologie



Testabdeckung

Kriterien

- Alle Anweisungen getestet
- Alle Entscheidungen getestet
- Alle Pfade getestet (*i.d.R. nicht möglich*)

Durchführung / Messung

- **Instrumentierung** des Quellcodes
- Zähler (initial 0, inkrementieren bei jedem Anweisungsbesuch, Ausgabe der Zählerstände)
- **Achtung:** durch Instrumentierung wird SUT modifiziert (beeinflusst bspw. Zeiteigenschaften)

Anweisungsüberdeckungstest

Prinzip des Anweisungsüberdeckungstests

Idee

- Sicherstellen, dass jede Anweisung mindestens einmal ausgeführt und getestet wird

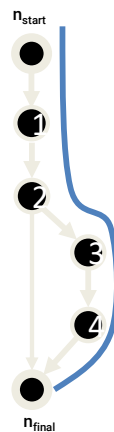
$$\text{Testabdeckung} = \frac{\text{Anzahl ausgeführter Anweisungen}}{\text{Anzahl aller Anweisungen}}$$

Synonyme

- Anweisungstest
- C_0 -Test

Beispiel

```
float getTax(float
1  income) {
    float taxRate =
      0.25,
2      taxAmount;
3
4  if (income >
    55000.0) {
      taxRate = 0.6;
```



Testfalldefinition

- Testfall 1: getTax(60000)
- Testabdeckung = 100%

Kritische Diskussion

- Anweisungsüberdeckung sehr schwaches Kriterium
- Fehler werden nicht zuverlässig erkannt
 - die nur auf bestimmten **Ausführungspfaden** auftreten
 - im Zusammenhang mit **zusammengesetzten Entscheidungen**
- Geringste Fehleridentifizierungsquote bei kontrollflussbasierten Tests (**nur 18%**)

Entscheidungsüberdeckungstest

Prinzip des Entscheidungsüberdeckungstests

Idee

- Sicherstellen, dass jede Verzweigung mindestens einmal ausgeführt und getestet wird

$$\text{Testabdeckung} = \frac{\text{Anzahl ausgeführter Zweige}}{\text{Anzahl aller Zweige}}$$

Synonyme

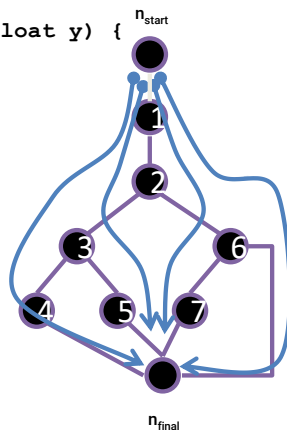
- Entscheidungstest
- Zweigtest oder Zweigüberdeckungstest
- C₁-Test

Beispiel

```

int getQuadrant(float x, float y) {
1  int quadrant = 1;
2  if (x < 0) {
3      if (y < 0)
4          quadrant = 3;
5      else
6          quadrant = 2;
7  }
8  else {
9      if (y < 0)
10         quadrant = 4;
11  }
12  return quadrant;
13 }

```

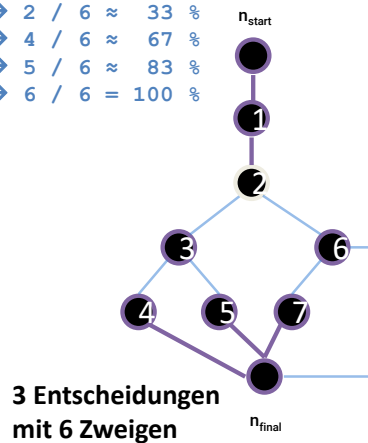


Testfalldefinition

- getQuadrant (-1, -1)
- getQuadrant (-1, 1)
- getQuadrant (1, -1)
- getQuadrant (1, 1)

Testabdeckung

- 1) `getQuadrant(-1, -1)` → 2 / 6 ≈ 33 %
- 2) `getQuadtant( 1, 1)` → 4 / 6 ≈ 67 %
- 3) `getQuadrant(-1, 1)` → 5 / 6 ≈ 83 %
- 4) `getQuadrant( 1, -1)` → 6 / 6 = 100 %



Testabdeckung

Beobachtung

- Bereits wenige Testfälle erzielen hohe Überdeckungsrate
- Mit jedem weiteren Testfall werden nur wenige neue Zweige durchlaufen

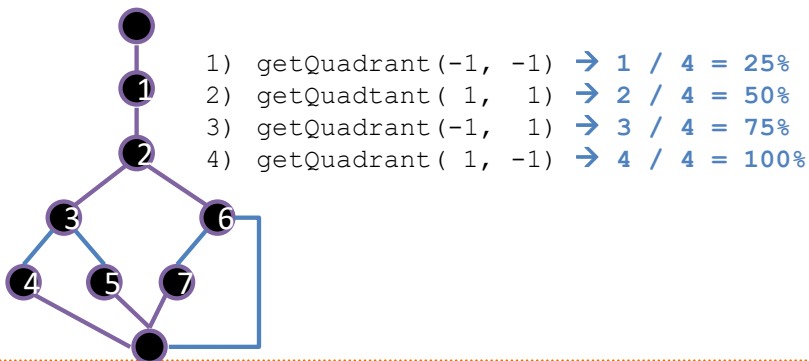
Problem

- **Vortäuschung falscher Sicherheit** bei wenigen Testfällen;
zum Test-Ende hin relativ viele Testfälle für **vollständige**
Zweigüberdeckung notwendig
(→ *Problem für das Testmanagement*)

Essentielle Zweigüberdeckung

Idee

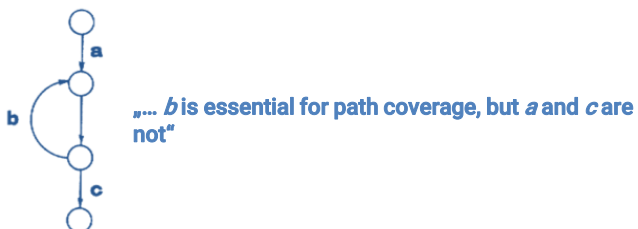
- Entscheidungen die immer getroffen werden müssen werden nicht gezählt



Essentielle Zweigüberdeckung

Schleifen

T. Chusho: Test Data Selection and Quality Estimation Based on the Concept of Essential Branches for Path Testing. In: *IEEE Transactions on Software Engineering*, Vol. 13, No. 5, May 1987, pp. 509 - 517.



Kritische Diskussion

- Fehleridentifizierungsquote von C_1 -Tests 34% (18% bei C_0)
- Entscheidungsüberdeckung ist minimales Testkriterium
- Fehler werden nicht zuverlässig erkannt
 - im Zusammenhang mit **mehrmaliger** Schleifenausführung
 - im Zusammenhang mit **zusammengesetzten Entscheidungen**